

Scenario 2: Brownfield, Add Click Analytics

Requirement

"Add click analytics to track URL usage."

Context

The core URL shortener (create, redirect, delete) already exists and is running. The task is to add analytics without breaking existing functionality and without adding latency to the redirect path, which is the highest-volume and most latency-sensitive operation in the system.

Brownfield Impact Analysis

Module	Change Type	Risk
ReadService	Modified, adds a Kafka publish call	Medium. Must not throw on Kafka failure.
domain/model	New entity <code>ClickEvent</code>	Low. Additive, no existing code modified.
domain/repository	New <code>ClickEventRepository</code>	Low. Additive.
kafka/	New package: Producer + Consumer	Low. New code.
api/controller	New <code>AnalyticsController</code>	Low. New endpoint, no existing routes changed.
service/	New <code>AnalyticsService</code>	Low. New code.
config/KafkaConfig	New file	Low. Spring auto-config handles the Kafka connection.
Database schema	New table <code>click_events</code>	Low. Hibernate auto-DDL adds the table without touching <code>short_urls</code> .

Modules NOT impacted

`WriteService`, `UrlController`, `ShortCodeGenerator`, `RedirectController` (delegates to `ReadService`), the `ShortUrl` entity, `ShortUrlRepository`, and, at this phase, the frontend beyond additive analytics pages.

Task Decomposition

Step 1: Data model. `ClickEvent` entity (`id`, `shortCode`, `clickedAt`, `ipAddress`, `referrer`, `userAgent`); `ClickEventRepository` with count/daily-aggregation/referrer-aggregation queries; verify Hibernate creates `click_events` without migrating `short_urls`.

Step 2: Kafka infrastructure. `ClickEventMessage` POJO; `KafkaConfig` declaring the `click-events` topic (3 partitions); `ClickEventProducer` (serialize, send, catch all exceptions); `ClickEventConsumer` (deserialize, save, log failures).

Step 3: Wire the producer into the read path. Inject `ClickEventProducer` into `ReadService`; publish after successful resolution. The try-catch lives in the producer, not `ReadService`, so the redirect can never fail because of analytics.

Step 4: Analytics API. `AnalyticsService.getAnalytics()`; `AnalyticsController` (GET `/api/v1/analytics/{code}`); `AnalyticsResponse` DTO.

Step 5: Frontend. `Analytics.tsx` page, an analytics link from the Manage page, `/analytics/:code` route.

Implementation Details

Producer design. The key constraint here is that an analytics failure must never cause a redirect failure.

```
try {
  kafkaTemplate.send(TOPIC, shortCode, json);
} catch (Exception e) {
  log.warn("Failed to publish click event for code={}: {}", shortCode, e.getMessage());
}
```

If Kafka is down, clicks are silently dropped and logged at WARN. That's an explicit trade-off: redirect availability over analytics completeness.

Message key strategy. Using `shortCode` as the Kafka key keeps all events for a URL on the same partition, which preserves temporal ordering and leaves room for future per-partition consumer state.

Consumer error handling. Failures are logged at ERROR and processing continues for subsequent messages. There's no DLQ in this prototype, that's covered in `RISKS_AND_TRADEOFFS.md`.

Aggregation queries run on demand with no pre-aggregation: daily clicks over the last 30 days, and top referrers. That's fine at prototype scale. A URL with millions of clicks would need pre-aggregation or a time-series store.

Acceptance Criteria

Criterion	Verified by
Clicking a short link increments click count	End-to-end via Docker Compose
Click data appears in analytics within seconds	Manual: redirect, then check <code>/api/v1/analytics/{code}</code>
Redirect succeeds even when Kafka is stopped	Manual: stop Kafka container, redirect, verify 302
Analytics endpoint returns <code>totalClicks</code> , <code>clicksByDay</code> , <code>topReferrers</code>	API response inspection
Existing POST/GET/DELETE endpoints unchanged	Re-run <code>UrlControllerIT</code>

Risk: Analytics Loss on Kafka Failure

Extended outages lose clicks during that window permanently in this prototype, though they're logged at WARN so operators have visibility. Production options: a fallback direct-DB write on publish failure (adds latency, ensures completeness), `acks=all` with retries (handles transient failures only), or a local buffer that drains on recovery. For a production-grade system, the fallback DB write is the right call. Accepting slightly higher redirect latency in exchange for complete analytics is a reasonable trade at that point.

Case B: Fixing Dead Integration Tests

A second brownfield case worth including here: a good illustration of Core Requirement #3 (codebase reasoning) applied to an existing codebase rather than one just written.

Symptom. `mvn test` reported `Tests run: 10`, matching the three unit test classes, but not `UrlControllerIT`, which the docs described as covering 4 more cases.

Impacted modules identified:

Module	Finding
pom.xml	No maven-failsafe-plugin bound. Surefire's default include pattern (**/*Test.java) doesn't match *IT.java, so the file was never compiled into the executed test set.
UrlControllerIT.java	Correctly written, correctly named by Failsafe convention, just never invoked.
application-test.yml	Once the IT test started running, a second issue appeared: spring.kafka.bootstrap-servers: localhost:9093 caused Spring Kafka's auto-configured KafkaAdmin and @KafkaListener container to attempt real broker connections during context startup, adding roughly 52 seconds of retry/timeout overhead per run despite KafkaTemplate being mocked.

Fix, scoped to minimize blast radius. Bind maven-failsafe-plugin in pom.xml (a test-only build concern with zero runtime impact); add spring.kafka.listener.auto-startup: false and tightened spring.kafka.admin.properties timeouts to application-test.yml only (test profile, zero production impact).

Validation. mvn verify before vs. after: 10 tests in about 60 seconds (the IT suite silently absent) became 14 tests in 20 seconds (10 unit plus 4 integration, all passing). Full traceability entry in AI_WORKFLOW.md.

This is the brownfield workflow in miniature. Read the actual build output rather than trusting the docs, trace the symptom to root cause across two files, make the smallest fix that addresses the root cause, and re-run the exact command that surfaced the problem to confirm.